



COMP S492F (2026)

Machine Learning

Project Report

Topic: Shoe Brand Classification

Student Name	Student ID
Ashwin Sundar	13139477
Anthony Jesu Ashok Savitha, Maria Dharshini	13126390

Introduction

This report documents the implementation of a deep learning image classification pipeline designed to distinguish three shoe brands: Nike, Adidas and Converse. The project follows a complete machine learning workflow including data preprocessing, transfer learning with EfficientNetB0, systematic training with validation and final model evaluation. The optimized model achieves a validation accuracy of 88.89% while maintaining the model size within 100 MB.

1. Data Preparation

Dataset Description

The provided dataset contains 600 training images and 90 validation images across three balanced classes (Nike, Adidas, Converse), all in 240×240 RGB .jpg format. The directory structure is:

```
data/
├── train/    (nike/, adidas/, converse/)
└── validate/ (nike/, adidas/, converse/)
```

Preprocessing & Augmentation Pipeline

- **Training Set:** Images are normalized using EfficientNetB0's `preprocess_input` function (scales pixels to $[-1, 1]$). Data augmentation was also applied to improve generalization:
 - Random rotation: $\pm 20^\circ$
 - Horizontal flip
 - Width/height shift: $\pm 10\%$
 - Zoom: $\pm 10\%$ (Fill mode: nearest)
- **Validation Set:** Only `preprocess_input` normalization is applied (no augmentation) to ensure realistic performance measurement.
- **Batch Loading:** `ImageDataGenerator` with `batch_size=32` and `class_mode='categorical'` for efficient memory utilization

2. Classification Implementation

Model Architecture

Transfer learning with EfficientNetB0 (ImageNet pre-trained) was selected for its parameter efficiency and strong feature extraction on small datasets. The backbone is frozen during initial training, and a custom classification head is appended:

Layer	Configuration	Purpose
EfficientNetB0	<code>include_top=False, pooling='avg'</code>	Feature extraction backbone

Dense	256 units, ReLU, L2(0.001)	High-level feature mapping
Dropout	Rate=0.5	Regularization for dense layer
Dense	128 units, ReLU, L2(0.001)	Feature refinement
Dropout	Rate=0.3	Additional regularization
Dense	3 units, Softmax	Final 3-class classification

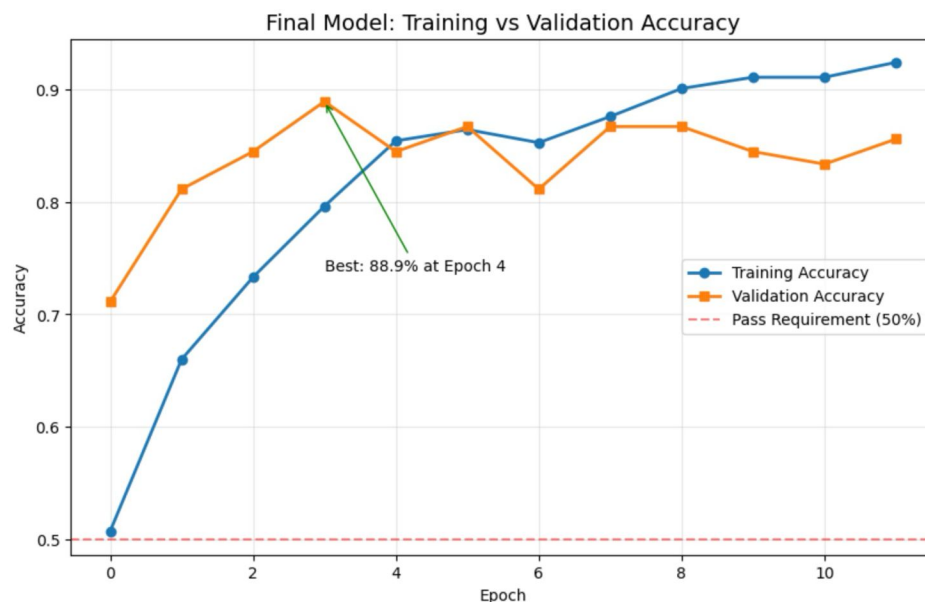
Training Configuration

- Loss function: Categorical Crossentropy (multi-class classification)
- Optimizer: Adam with initial learning rate = 0.001
- Metrics: Accuracy
- Callbacks:
 - ModelCheckpoint: Saves best validation accuracy weight
 - EarlyStopping: Halts training if val_accuracy stagnates for 8 epochs (restores best weights)
 - ReduceLROnPlateau: Reduces LR by $0.5 \times$ if val_loss plateaus for 3 epochs (min_lr = $1e-6$)

3. Training Outcome & Results

Training stopped early at epoch 12 and the best validation accuracy of 88.9% was achieved at epoch 4. The moderate gap between training (~92%) and validation accuracy indicates strong generalization without severe overfitting.

Accuracy Plot:



Key Observations

- Validation accuracy >80% within the first 4 epochs due to strong ImageNet feature transfer
- Dropout + L2 regularization effectively mitigated overfitting on the limited dataset (600 training samples)
- ReduceLROnPlateau enabled fine-grained convergence, preventing oscillation near optimal minima

4. Model Testing and Reloading Instructions

Prerequisites

- Python 3.10+, TensorFlow ≥ 2.12
- Model file: shoe_classifier_model.keras (included in submission)
- Test dataset structured identically to training/validation sets (test_data/nike/, test_data/adidas/, test_data/converse/)

(Included in the Notebook ML_miniProject.ipynb)

```
import tensorflow as tf
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.image import ImageDataGenerator

#1.Load model
model = load_model('shoe_classifier_model.keras')

#2.Recompile (required for metrics)
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

#3.Prepare test generator with matching preprocessing
test_datagen = ImageDataGenerator(
    preprocessing_function=tf.keras.applications.efficientnet.preprocess_input
)
test_gen = test_datagen.flow_from_directory(
    'path/to/test_data',          # Replace with actual path
    target_size=(240, 240),
    batch_size=32,
    class_mode='categorical',
    shuffle=False                  # Critical for deterministic evaluation
)

# 4. Reset generator & evaluate
test_gen.reset()
loss, accuracy = model.evaluate(test_gen, verbose=1)
print(f"Test Accuracy: {accuracy:.4f}")
```

Conclusion

A transfer learning model based on EfficientNetB0 successfully classifies three shoe brands with high accuracy (88.89%). The implementation includes data preprocessing, validation, model saving, and a reloading test cell. The model size (20 MB) is well within the 100 MB submission limit.

Appendix

The following table records different hyperparameter configurations tested and their corresponding training and validation accuracies:

Run	Learning Rate	Batch Size	Epochs	Regularization	Train Acc	Val Acc
1	0.001	32	15	Dropout: 0.3	97.0%	88.9%
2	0.001	32	25	Dropout: 0.5 + L2: 0.001	96.0%	87.8%
3	0.0001	32	25	Dropout: 0.5 + L2: 0.001	86.6%	84.4%

* Note: Learning rate was reduced automatically by ReduceLROnPlateau during training.

Tools and Libraries

The following table summarises the key libraries and modules used throughout the project, along with their primary roles in the implementation pipeline.

Library	Primary Use
TensorFlow / Keras	Model building, training, evaluation, and data preprocessing
NumPy	Array operations and label processing
Matplotlib	Accuracy and loss plots (required by project)
Scikit-learn	Classification metrics (precision, recall, F1-score)
Seaborn	Confusion matrix heatmap visualisation
Pathlib	Cross-platform file path handling
OS	File system operations (e.g., checking model file size)
Google Colab	Mounting Google Drive and downloading output files